

Project Architecture Blueprint

Repository: UPSProd4/P4AG_Operations_Technology/era-da-dcms-OverGoods

Generated: 2026-07-13T11:19:58.207-04:00

Revised after adversarial review: 2026-07-13

Blueprint intent: Definitive architecture reference for maintaining consistency while legacy functionality is migrated to the new service folders.

0. DOCUMENT CONVENTIONS AND EVIDENCE MODEL

This blueprint now separates observed current-state facts from target-state guidance.

| Tag | Meaning |

|---|---|

| **[Detected]** | Directly evidenced from repository artifacts listed in this document. |

| **[Observed]** | Reverse-engineered interpretation from implementation patterns. |

| **[Proposed]** | Forward-looking recommendation for the migration target state. |

Section 15 is intentionally **not** treated as ratified ADR history; it is a reverse-engineered record for discussion.

1. ARCHITECTURE DETECTION AND ANALYSIS

Technology stack (detected)

| Area | Primary tech | Evidence |

|---|---|---|

| Current production app | ASP.NET WebForms on .NET Framework 4.6.2 |

Legacy\DI_Overgoods\OGSRoot\Overgoods\web.config , **OGS.sln** website project
Overgoods_SSO |

| Shared domain/integration libraries | Mixed C# + VB.NET class libraries |

OGSLib\Libraries*.csproj , ***.vbproj** |

| AJAX service endpoints | ASMX ScriptServices | **Overgoods\OGSService.asmx** ,

ProductCodeProcessor.asmx |

| Data access | SQL Server + stored procedures | **SqlCommand.CommandType =**

StoredProcedure across libraries |

| Authentication | Azure AD OAuth/OpenID + MSAL + legacy session state |

Overgoods\login.aspx.cs , **App_Code\TokenValidator.cs** |

| Auxiliary runtime services | Windows/console services (.NET Framework 4.5.2-4.7.2), WCF self-host |

OvergoodsToCipe , **OGSNewTrackingService** , **OGSModernBrowserPrintService** |

| Target-state rewrite scaffolds | Service folders exist but are still scaffolds | **api-service** , **business-**
service , **data-service** , **retry-service** , **ui-service** (README/.gitignore only) |

Architectural pattern

- **[Detected] Current:** Legacy modular monolith (WebForms + shared libraries) with satellite integration services.

- **[Proposed] Target:** Service-oriented modernization, pending explicit contract and boundary definition.

2. ARCHITECTURAL OVERVIEW

The production path is a **single IIS-hosted WebForms application** (**Overgoods**) coordinating domain logic through **OGSLib** libraries and SQL stored procedures in the **Overgoods** database. The solution is modular by library boundaries, but runtime deployment is primarily monolithic.

The repository also contains standalone executables/services for integrations (Clpe export, CEC tracking updates, warehouse alignment file generation, and browser print service), forming a hybrid topology: monolithic core + peripheral integration processes.

3. NON-FUNCTIONAL REQUIREMENTS BASELINE (PROPOSED)

The original blueprint had no NFR targets. The following baseline must be refined with production telemetry before target-state sign-off:

- **Availability target:** 99.9% monthly for operator-critical workflows.
- **Latency target:** p95 under 2 seconds for interactive inventory/search actions.
- **Recovery objectives:** RTO <= 4 hours, RPO <= 15 minutes for operational data.
- **Resilience target:** external dependency outages degrade to bounded failure modes, not global outage.
- **Scalability target:** publish and maintain expected concurrent operator and transaction volume envelopes.

4. ARCHITECTURE VISUALIZATION

High-level context (system context view)

flowchart LR

U[Warehouse/User Operators] --> W[Overgoods WebForms App\nIIS / Overgoods_SS0]

W --> L[OGSLib Domain Libraries\nC# + VB.NET]

L --> DB[(SQL Server\nOvergoods + ARES_SEARCH)]

W --> AD[Azure AD / MSAL]

W --> GCP[Google Cloud Storage]

W --> EXT[External UPS/Partner APIs\n(eTT, SCS, tracking)]

```
DB --> C1[OvergoodsToCipe Service\nActiveMQ producer]

DB --> C2[OGSNewTrackingService\nCEC claim update API]

DB --> C3[WarehouseAlignment job\nfile drop output]

W --> C4[ModernBrowserPrintService\nlocal print host]
```

Runtime component view

flowchart TD

```
P[WebForms Pages (*.aspx/.aspx.cs)] --> S[SessionManager + BasePage + App_Code]
```

```
P --> A1[OGSService.asmx]
```

```
P --> A2[ProductCodeProcessor.asmx]
```

```
A1 --> D1[InventoryLibrary]
```

```
A1 --> D2>ReturnsLibrary]
```

```
A1 --> D3[OGSLibrary]
```

```
P --> D4[UserLibrary / WarehouseLibrary / CommonLibrary]
```

```
D1 --> SQL[(Stored Procedures)]
```

```
D2 --> SQL
```

```
D3 --> SQL
```

```
D4 --> SQL
```

5. CORE ARCHITECTURAL COMPONENTS

A. Web presentation layer (Legacy\DI_Overgoods\OGSRoot\Overgoods) [Detected]

- 100 WebForms pages with code-behind (`*.aspx.cs`), master page, user controls.
- Responsibility: workflow orchestration, UI event handling, page-level validation, redirect/authorization behavior.
- Interaction: direct calls into domain libraries and ASMX endpoints.

B. App_Code application services [Detected]

- Central classes: `SessionManager` , `TokenValidator` , `DbAccess` , `ProductCodeProcessor` , `GcsHelper` , `CustomHttpEncoder` .
- Responsibility: request/session state, token refresh, helper-level data access, UPC/NDC lookup processing, GCS integration.

C. Domain/data libraries (`OGSLib\Libraries`) [Detected]

- VB: `OGSLibrary` (`AresLibrary`) , `CommonLibrary` , `WarehouseLibrary` .
- C#: `InventoryLibrary` , `ReturnsLibrary` , `UserLibrary` , `HotListLibrary` , `eTTInterface` , `clsUPSShipAPI` , `PrintDataSetsLibrary` .
- Responsibility: domain operations, report/data retrieval, external protocol mapping, shared constants/utilities.

D. Integration services (standalone) [Detected]

- `OvergoodsToCipe` : extracts DB events, serializes to JSON, publishes to ActiveMQ.
- `OGSNewTrackingService` : token retrieval + claim tracking API update job.
- `OGSModernBrowserPrintService` : local WCF/WebHttp print endpoint.
- `OGSWarehouseAlignment` : scheduled file generation from DB state.

E. Data layer [Detected]

- SQL Server schemas/scripts in `Legacy\DI_Overgoods\Databases` :
 - `create-overgoods.sql`
 - `create-ARES_SEARCH.sql`
 - `Overgoods-script.sql`
 - `ares_search-script.sql`

6. ARCHITECTURAL LAYERS AND DEPENDENCIES

1. **UI/Endpoint Layer:** WebForms pages + ASMX.
2. **Application/Session Layer:** `App_Code` orchestration and state.
3. **Domain/Business Layer:** `OGSLib\Libraries` .
4. **Infrastructure/Data Layer:** SQL stored procedures, external services, file outputs.

[Detected] Dependency direction is largely top-down, with known layer skips (for example direct UI-to-data helper paths).

[Proposed] New implementations should enforce strict dependency direction and explicitly track exceptions.

7. DATA ARCHITECTURE

Current state (detected)

- Dominant access style: stored procedures (`SqlCommand.CommandType = StoredProcedure`).
- Connection model: shared connection string from `CommonLibrary.Constants.OVERGOODS` .
- Data shape: `DataTable` , `DataSet` , custom POJO/DTO objects.
- Session data model: per-user operational state via SQL-backed ASP.NET session state (`web.config`).
- Reporting/export: DB queries transformed into CSV/RTF/JSON/event payloads.

Database topology (detected)

- **Overgoods DB:** primary operational workflows.
- **ARES_SEARCH DB:** present in scripts and architecture references; requires explicit ownership and usage documentation before migration design is finalized.

Target-state data ownership and consistency model (proposed)

- Define a per-service ownership matrix before any service-folder implementation.
- Ban direct cross-service writes to another service's owned schema.
- For cross-boundary workflows, use idempotency + outbox/event patterns rather than ad hoc distributed transactions.
- Define read-model and freshness expectations for eventual-consistency paths.

8. CROSS-CUTTING CONCERNS IMPLEMENTATION

Authentication and authorization

- **[Detected]** Azure AD OAuth flow in `login.aspx.cs` .
- **[Detected]** Token lifecycle with session cache in `TokenValidator` .
- **[Detected]** Legacy manager override path in `auth.aspx.cs` .
- **[Proposed]** Document manager override guardrails explicitly: trigger conditions, approved roles, audit logging, production enablement policy, and removal/deprecation plan if applicable.

Security architecture controls

- **[Detected]** Anti-forgery checks, custom request encoder, hardened headers.
- **[Proposed]** Add trust-boundary map and threat model that includes:
 - external API trust boundaries,
 - service-to-service authentication expectations,
 - token scope/lifetime policy,
 - secret storage and rotation standards.

Error handling

- **[Detected]** `CommonLibrary.Logging.LogException(...)` exists as common entry point.
- **[Detected]** Mixed handling strategy (propagate vs swallow/default payloads).
- **[Proposed]** Define explicit error contract policy by boundary (UI/API/worker) to reduce inconsistent behavior.

Logging and observability

- **[Detected]** DB-backed exception publishing and local service loggers.
- **[Proposed]** Baseline requirements for migration:
 - structured log schema,
 - correlation IDs across service boundaries,
 - SLI/SLO definitions aligned to Section 3 NFRs,
 - alert ownership and runbooks for critical workflows.

Configuration management

- **[Detected]** `web.config` + `App.config` endpoint/auth/SMTP/proxy/print keys.
- **[Observed]** Host checks + config keys currently drive environment behavior.
- **[Proposed]** Define centralized target-state configuration ownership and secret boundary rules.

9. SERVICE COMMUNICATION PATTERNS

- **Synchronous HTTP (intra-app):** browser -> WebForms/ASMX.
- **Synchronous HTTP (external):** OAuth token, external tracking/update APIs.
- **Message queue (async):** `OvergoodsToCipe` -> ActiveMQ (`QueueSender`).
- **File-based integration:** warehouse alignment output files for downstream systems.
- **Local service call:** web app to browser print host.

10. FAILURE MODES AND RESILIENCE (PROPOSED)

| Dependency | Failure mode | Required behavior |

|---|---|---|

| SQL Server (`Overgoods` , `ARES_SEARCH`) | timeout / failover / lock contention | bounded retries with timeout budget, degrade non-critical screens, alert DB ops with correlation ID |

| Azure AD / token endpoints | token issue/refresh failure | fail closed for privileged operations, preserve user-facing retry path, alert on repeated auth failures |

| External APIs (tracking/eTT/SCS) | latency / 5xx / schema drift | circuit-breaker + retry policy + dead-letter handling where asynchronous |

| ActiveMQ publish path | broker unavailable / queue backlog | outbox buffering, retry with backoff, operator alert on queue lag thresholds |

| Print host | local service unavailable | clear operator fallback and reprint recovery path |

| File-drop integrations | downstream pickup failure | file generation audit trail + re-drive mechanism + alerting on stale outbound artifacts |

11. TECHNOLOGY-SPECIFIC ARCHITECTURAL PATTERNS

.NET Framework / WebForms patterns [Detected]

- Code-behind page controllers.
- ASMX ScriptService endpoints returning script/serialized payloads.
- Library-centric business logic with static managers and utility modules.
- Classic `packages.config` restore model.

Data access patterns [Detected]

- ADO.NET (`SqlConnection` , `SqlCommand` , `SqlDataAdapter`).
- Stored-procedure-driven contracts.
- Low use of ORM abstractions.

12. IMPLEMENTATION PATTERNS

Interface and boundary patterns [Detected]

- Interface usage appears in targeted integration areas (for example queue sender abstractions), not as a consistent domain-wide DI contract.

Service implementation template (legacy style)

```
[WebMethod(EnableSession = true)]

public string LookupProductCode(string productCode, string userID)

{

    LookupRequest req = new LookupRequest(productCode, userID);

    LookupResponse resp = FollowProcessPath(req,
XLib.ProductProcessingUtils.GetUpcLookupProcess());

    return resp.LookupOutput.ToString();

}
```

Repository/data access template (legacy style)

```
using (SqlCommand cmd = new SqlCommand("dbo.usp_Name", connection))

{

    cmd.CommandType = CommandType.StoredProcedure;

    // add parameters

}
```

```
}
```

13. TESTING ARCHITECTURE

Current state (detected)

- No dedicated automated unit/integration architecture in primary legacy solution (`OGS.sln`).
- Quality assurance is primarily environment validation + operational/report verification.

Target-state testing baseline (proposed)

- **Unit tests:** business rules and policy logic in migrated services.
- **Contract tests:** API boundary and service integration contracts.
- **Integration tests:** data adapters and critical external dependency paths.
- **Characterization tests:** lock down legacy behavior before migration cutover.
- **CI gate:** require test execution for any new service-folder production behavior.

14. DEPLOYMENT ARCHITECTURE

- **[Detected]** IIS-hosted WebForms site (`Overgoods_SSO`) on .NET Framework 4.6.2.
- **[Detected]** SQL Server session persistence.
- **[Detected]** Supporting Windows jobs/services for print, queue export, tracking updates, alignment file generation.
- **[Detected]** Build model is Visual Studio/MSBuild classic solution files.
- **[Proposed]** Define explicit coexistence deployment topology before feature migration milestones.

15. EXTENSION AND EVOLUTION PATTERNS

Feature addition (current state)

1. Add/extend WebForms page + code-behind.
2. Add/update library manager methods.
3. Add stored procedures and DB script updates.
4. Wire logging, session state, and config keys.

Target-state migration pattern (proposed, pending ADR sign-off)

1. Define domain boundary and contract first (do not start from folder name alone).
2. Implement API boundary with explicit request/response/error semantics.
3. Place business logic behind explicit interfaces; do not couple UI directly to data adapters.
4. Implement data ownership and consistency policy from Section 7.
5. Add observability and failure-mode handling from Sections 8 and 10 before production rollout.

Coexistence, routing, and rollback requirements (proposed)

- Define route ownership during hybrid operation (legacy vs new service path).
- Require feature-flag or routing-switch controls for cutover.

- Define rollback trigger conditions and rollback operator runbook per migrated feature.
- Define service-level done criteria and legacy decommission criteria.

16. ARCHITECTURAL EXAMPLES

Layer separation example

- UI event handlers call domain library managers (`InventoryManager` , `ReturnsManager`) rather than embedding SQL in most page code.

Component communication example

- `OGSService` and `ProductCodeProcessor` expose session-enabled AJAX methods consumed by pages for dynamic workflows.

Extension point example

- Product lookup process path selection (`XLib.ProductProcessingUtils.GetUpcLookupProcess`) allows environment-driven source sequencing.

17. OBSERVED HISTORICAL DECISIONS (REVERSE-ENGINEERED, NOT RATIFIED ADRS)

OBS-001: Shared-library concentration of business logic

- **Context:** Large WebForms surface area with repeated workflows.
- **Observed decision:** Reusable behavior concentrated in `OGSLib\Libraries` .
- **Consequence:** Reuse improved, but coupling to static/global patterns increased.

OBS-002: Stored-procedure-centric data contract

- **Context:** Operational DB rules and reporting dependence.
- **Observed decision:** SPs used for most write/read operations.
- **Consequence:** Stable DB contract; harder local unit testing and schema discovery.

OBS-003: Hybrid modernization approach

- **Context:** Need incremental migration without operational shutdown.
- **Observed decision:** Legacy app remains active while new service folders are introduced.
- **Consequence:** Parallel architecture governance is mandatory to prevent drift.

18. ARCHITECTURE GOVERNANCE

Current governance signals (detected)

- Solution-level project boundaries.
- Shared constants/logging modules.
- DB script artifacts.

Gaps (detected)

- No automated architecture conformance checks.
- Limited test enforcement.
- Mixed exception handling patterns.

Enforceable controls (proposed)

- Mandatory ADR workflow for target-state boundary changes.
- CODEOWNERS protections for `Legacy\` and top-level service folders.
- CI policy checks for forbidden patterns (for example net-new legacy feature paths without approved exception).
- Time-bound exception process with explicit approver and expiration.

19. BLUEPRINT FOR NEW DEVELOPMENT

Development workflow by feature type (proposed default, subject to domain-boundary ADR)

| *Feature type* | *Start point* | *Primary placement* |

---|---|---

| New API capability | Domain-boundary decision + API contract | API endpoint + contract tests |

| Business rule change | Domain/service owner | Domain logic + unit tests |

| Data/query behavior | Data owner service | Data adapters + integration tests |

| Retry/background reliability | Workflow owner | Durable worker/outbox/consumer implementation |

| UI/UX changes | `ui-service` | React/Vite components + API clients |

Distributed-monolith risk guardrail (proposed)

- Do not assume `api-service` / `business-service` / `data-service` are separate deployables by default.
- Require explicit ADR comparing horizontal split vs domain-vertical slices.
- If horizontal layering is retained, document expected deployment coupling and blast-radius trade-offs.

VB.NET disposition (proposed)

- Before migrating business logic, choose and document one path:
 1. Port VB.NET libraries to C# in target runtime.
 2. Wrap VB.NET logic behind compatibility adapters temporarily.
 3. Maintain .NET Framework shim with explicit sunset timeline.

Common pitfalls to avoid

- Reintroducing direct SQL access in UI/API layers.
- Adding new logic to dated snapshot folders under `Legacy\DI_Overgoods\OGS_PROD` or `OGSRoot_Production_2019-03-28`.
- Duplicating domain rules across WebForms and new services.
- Silent catch blocks that hide integration failures.

20. UPDATE GUIDANCE

- Regenerate this blueprint after major schema changes, service-folder implementation milestones, or cross-cutting security/auth changes.
- Maintain a changelog section once migration implementation becomes active in top-level service folders.
- Keep **Detected** , **Observed** , and **Proposed** labels accurate on each revision.